Revenue Data and Building a Dashboard

November 7, 2026

Extracting and Visualizing Stock Data

Description

Extracting essential data from a dataset and displaying it is a necessary part of data science; therefore individuals can make correct decisions based on the data. In this assignment, you will extract some stock data, you will then display this data in a graph.

Table of Contents

    Define a Function that Makes a Graph
    Question 1: Use yfinance to Extract Stock Data
    Question 2: Use Webscraping to Extract Tesla Revenue Data
    Question 3: Use yfinance to Extract Stock Data
    Question 4: Use Webscraping to Extract GME Revenue Data
    Question 5: Plot Tesla Stock Graph
    Question 6: Plot GameStop Stock Graph

Estimated Time Needed: 30 min

Note:- If you are working Locally using anaconda, please uncomment the following code and execute it. Use the version as per your python version.

[79]:
```
!pip install yfinance
!pip install bs4
!pip install nbformat
!pip install matplotlib
```

```
Requirement already satisfied: yfinance in /opt/conda/lib/python3.12/site-
packages (1.8.0)
Requirement already satisfied: beautifulsoup4>=4.11.1 in
/opt/conda/lib/python3.12/site-packages (from yfinance) (4.13.4)
Requirement already satisfied: curl_cffi>=0.17 in
/opt/conda/lib/python3.12/site-packages (from yfinance) (0.17.0)
Requirement already satisfied: lxml>=4.9.0 in /opt/conda/lib/python3.12/site-
packages (from yfinance) (6.1.3)
Requirement already satisfied: numpy>=1.16.5 in /opt/conda/lib/python3.12/site-
packages (from yfinance) (2.5.4)
Requirement already satisfied: pandas>=1.3.0 in /opt/conda/lib/python3.12/site-
packages (from yfinance) (3.1.1)
```

```
Requirement already satisfied: peewee>=3.16.2 in /opt/conda/lib/python3.12/site-
packages (from yfinance) (4.5.3)
Requirement already satisfied: platformdirs>=2.0.0 in
/opt/conda/lib/python3.12/site-packages (from yfinance) (4.3.7)
Requirement already satisfied: protobuf>=3.19.0 in
/opt/conda/lib/python3.12/site-packages (from yfinance) (7.36.2)
Requirement already satisfied: pytz>=2022.5 in /opt/conda/lib/python3.12/site-
packages (from yfinance) (2025.2)
Requirement already satisfied: requests>=2.31 in /opt/conda/lib/python3.12/site-
packages (from yfinance) (2.32.3)
Requirement already satisfied: websockets>=13.0 in
/opt/conda/lib/python3.12/site-packages (from yfinance) (17.3)
Requirement already satisfied: soupsieve>1.2 in /opt/conda/lib/python3.12/site-
packages (from beautifulsoup4>=4.11.1->yfinance) (2.5)
Requirement already satisfied: typing-extensions>=4.0.0 in
/opt/conda/lib/python3.12/site-packages (from beautifulsoup4>=4.11.1->yfinance)
(4.16.0)
Requirement already satisfied: cffi>=2.1.1 in /opt/conda/lib/python3.12/site-
packages (from curl_cffi>=0.17->yfinance) (2.1.1)
Requirement already satisfied: certifi>=2026.10.19 in
/opt/conda/lib/python3.12/site-packages (from curl_cffi>=0.17->yfinance)
(2026.10.23)
Requirement already satisfied: pycparser in /opt/conda/lib/python3.12/site-
packages (from cffi>=2.1.1->curl_cffi>=0.17->yfinance) (2.22)
Requirement already satisfied: python-dateutil>=2.9.0 in
/opt/conda/lib/python3.12/site-packages (from pandas>=1.3.0->yfinance)
(2.9.0.post0)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-
packages (from python-dateutil>=2.9.0->pandas>=1.3.0->yfinance) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/opt/conda/lib/python3.12/site-packages (from requests>=2.31->yfinance) (3.4.2)
Requirement already satisfied: idna<4,>=2.5 in /opt/conda/lib/python3.12/site-
packages (from requests>=2.31->yfinance) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/opt/conda/lib/python3.12/site-packages (from requests>=2.31->yfinance) (2.4.0)
Requirement already satisfied: bs4 in /opt/conda/lib/python3.12/site-packages
(0.0.2)
Requirement already satisfied: beautifulsoup4 in /opt/conda/lib/python3.12/site-
packages (from bs4) (4.13.4)
Requirement already satisfied: soupsieve>1.2 in /opt/conda/lib/python3.12/site-
packages (from beautifulsoup4->bs4) (2.5)
Requirement already satisfied: typing-extensions>=4.0.0 in
/opt/conda/lib/python3.12/site-packages (from beautifulsoup4->bs4) (4.16.0)
Requirement already satisfied: nbformat in /opt/conda/lib/python3.12/site-
packages (5.10.4)
Requirement already satisfied: fastjsonschema>=2.15 in
/opt/conda/lib/python3.12/site-packages (from nbformat) (2.21.1)
Requirement already satisfied: jsonschema>=2.6 in
```

```
/opt/conda/lib/python3.12/site-packages (from nbformat) (4.23.0)
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in
/opt/conda/lib/python3.12/site-packages (from nbformat) (5.7.2)
Requirement already satisfied: traitlets>=5.1 in /opt/conda/lib/python3.12/site-
packages (from nbformat) (5.14.3)
Requirement already satisfied: attrs>=22.2.0 in /opt/conda/lib/python3.12/site-
packages (from jsonschema>=2.6->nbformat) (25.3.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
/opt/conda/lib/python3.12/site-packages (from jsonschema>=2.6->nbformat)
(2025.4.1)
Requirement already satisfied: referencing>=0.28.4 in
/opt/conda/lib/python3.12/site-packages (from jsonschema>=2.6->nbformat)
(0.36.2)
Requirement already satisfied: rpds-py>=0.7.1 in /opt/conda/lib/python3.12/site-
packages (from jsonschema>=2.6->nbformat) (0.24.0)
Requirement already satisfied: platformdirs>=2.5 in
/opt/conda/lib/python3.12/site-packages (from jupyter-
core!=5.0.*,>=4.12->nbformat) (4.3.7)
Requirement already satisfied: typing-extensions>=4.4.0 in
/opt/conda/lib/python3.12/site-packages (from
referencing>=0.28.4->jsonschema>=2.6->nbformat) (4.16.0)
Requirement already satisfied: matplotlib in /opt/conda/lib/python3.12/site-
packages (3.11.2)
Requirement already satisfied: contourpy>=1.0.1 in
/opt/conda/lib/python3.12/site-packages (from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /opt/conda/lib/python3.12/site-
packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in
/opt/conda/lib/python3.12/site-packages (from matplotlib) (4.68.0)
Requirement already satisfied: kiwisolver>=1.3.1 in
/opt/conda/lib/python3.12/site-packages (from matplotlib) (1.5.1)
Requirement already satisfied: numpy>=1.25 in /opt/conda/lib/python3.12/site-
packages (from matplotlib) (2.5.4)
Requirement already satisfied: packaging>=20.0 in
/opt/conda/lib/python3.12/site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=9 in /opt/conda/lib/python3.12/site-
packages (from matplotlib) (13.0.0)
Requirement already satisfied: pyparsing>=3 in /opt/conda/lib/python3.12/site-
packages (from matplotlib) (3.3.3)
Requirement already satisfied: python-dateutil>=2.7 in
/opt/conda/lib/python3.12/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-
packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

[80]:
```
import yfinance as yf
import pandas as pd
import requests
```

```
from bs4 import BeautifulSoup
```

In Python, you can ignore warnings using the warnings module. You can use the filterwarnings function to filter or ignore specific warning messages or categories.

[81]:
```
import warnings
# Ignore all warnings
warnings.filterwarnings("ignore", category=FutureWarning)
```

0.1 Define Graphing Function

In this section, we define the function `make_graph`. **You don't have to know how the function works, you should only care about the inputs. It takes a dataframe with stock data (dataframe must contain Date and Close columns), a dataframe with revenue data (dataframe must contain Date and Revenue columns), and the name of the stock.**

[82]:
```
# The make_graph function has been modified to use Matplotlib for static graphs.
 ↪ Earlier, it used Plotly to generate interactive dashboards, which caused␣
 ↪issues when uploading the notebook in the MARK assignment submission.



import matplotlib.pyplot as plt

def make_graph(stock_data, revenue_data, stock):
    stock_data_specific = stock_data[stock_data.Date <= '2021-06-14']
    revenue_data_specific = revenue_data[revenue_data.Date <= '2021-04-30']

    fig, axes = plt.subplots(2, 1, figsize=(12, 8), sharex=True)

    # Stock price
    axes[0].plot(pd.to_datetime(stock_data_specific.Date), stock_data_specific.
 ↪Close.astype("float"), label="Share Price", color="blue")
    axes[0].set_ylabel("Price ($US)")
    axes[0].set_title(f"{stock} - Historical Share Price")

    # Revenue
    axes[1].plot(pd.to_datetime(revenue_data_specific.Date),␣
 ↪revenue_data_specific.Revenue.astype("float"), label="Revenue",␣
 ↪color="green")
    axes[1].set_ylabel("Revenue ($US Millions)")
    axes[1].set_xlabel("Date")
    axes[1].set_title(f"{stock} - Historical Revenue")

    plt.tight_layout()
    plt.show()
```

Use the make_graph function that we've already defined. You'll need to invoke it in questions 5

and 6 to display the graphs and create the dashboard. > **Note: You don't need to redefine the function for plotting graphs anywhere else in this notebook; just use the existing function.**

0.2 Question 1: Use yfinance to Extract Stock Data

Using the `Ticker` function enter the ticker symbol of the stock we want to extract data on to create a ticker object. The stock is Tesla and its ticker symbol is `TSLA`.

[83]:
```
tesla = yf.Ticker("TSLA")
print(tesla)
```

```
yfinance.Ticker object <TSLA>
```

Using the ticker object and the function `history` extract stock information and save it in a dataframe named `tesla_data`. Set the `period` parameter to `"max"` so we get information for the maximum amount of time.

[84]:
```
tesla_data = tesla.history(period="max")
tesla_data
```

[84]:

Date	Open	High	Low	Close	Volume	Dividends	Stock Splits
2010-06-29 00:00:00-04:00	1.266667	1.666667	1.169333	1.592667	281494500	0.0	0.0
2010-06-30 00:00:00-04:00	1.719333	2.028000	1.553333	1.588667	257806500	0.0	0.0
2010-07-01 00:00:00-04:00	1.666667	1.728000	1.351333	1.464000	123282000	0.0	0.0
2010-07-02 00:00:00-04:00	1.533333	1.540000	1.247333	1.280000	77097000	0.0	0.0
2010-07-06 00:00:00-04:00	1.333333	1.333333	1.055333	1.074000	103003500	0.0	0.0
…	…	…	…	…	…	…	…
2026-11-02 00:00:00-05:00	404.660004	414.230011	397.609985	403.820007	43141100	0.0	0.0
2026-11-03 00:00:00-05:00	406.250000	406.500000	396.529999	398.500000	26647700	0.0	0.0
2026-11-04 00:00:00-05:00	379.799988	392.040009	375.500000	385.190002	47805800	0.0	0.0
2026-11-05 00:00:00-05:00	374.119995	374.299988	338.149994	343.690002	79675500	0.0	0.0
2026-11-06 00:00:00-05:00	348.920013	363.839996	347.600006	358.220001	36642300	0.0	0.0

```
[4116 rows x 7 columns]
```

Reset the index using the `reset_index(inplace=True)` function on the tesla_data DataFrame and display the first five rows of the `tesla_data` dataframe using the `head` function. Take a screenshot of the results and code from the beginning of Question 1 to the results below.

[85]:
```
tesla_data.reset_index(inplace=True)
tesla_data.head()
```

```
/tmp/ipykernel_299/3012578559.py:1: Pandas4Warning: The inplace keyword in
DataFrame.reset_index is deprecated and will be removed in a future version
(PDEP-8).
See https://pandas.pydata.org/docs/dev/whatsnew/v3.1.0.html#deprecation-inplace
for more details.
  tesla_data.reset_index(inplace=True)
```

[85]:

	Date	Open	High	Low	Close	Volume	Dividends	Stock Splits
0	2010-06-29 00:00:00-04:00	1.266667	1.666667	1.169333	1.592667	281494500	0.0	0.0
1	2010-06-30 00:00:00-04:00	1.719333	2.028000	1.553333	1.588667	257806500	0.0	0.0
2	2010-07-01 00:00:00-04:00	1.666667	1.728000	1.351333	1.464000	123282000	0.0	0.0
3	2010-07-02 00:00:00-04:00	1.533333	1.540000	1.247333	1.280000	77097000	0.0	0.0
4	2010-07-06 00:00:00-04:00	1.333333	1.333333	1.055333	1.074000	103003500	0.0	0.0

0.3 Question 2: Use Webscraping to Extract Tesla Revenue Data

Use the `requests` library to download the webpage https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-PY0220EN-SkillsNetwork/labs/project/revenue.htm Save the text of the response as a variable named `html_data`.

[86]:
```
url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/
↪IBMDeveloperSkillsNetwork-PY0220EN-SkillsNetwork/labs/project/revenue.htm"
html_data = requests.get(url).text
#print(html_data)
```

Parse the html data using `beautiful_soup` using parser i.e `html5lib` or `html.parser`.

[87]:
```
soup = BeautifulSoup(html_data,'html.parser')
```

Using `BeautifulSoup` or the `read_html` function extract the table with `Tesla Revenue` and store it into a dataframe named `tesla_revenue`. The dataframe should have columns `Date` and `Revenue`.

Step-by-step instructions

Here are the step-by-step instructions:

1. Create an Empty DataFrame
2. Find the Relevant Table
3. Check for the Tesla Quarterly Revenue Table
4. Iterate Through Rows in the Table Body
5. Extract Data from Columns
6. Append Data to the DataFrame

Click here if you need help locating the table

Below is the code to isolate the table, you will now need to loop through the rows and columns

```
soup.find_all("tbody")[1]
```

If you want to use the read_html function the table is located at index 1

We are focusing on quarterly revenue in the lab.

[88]:
```
tesla_revenue = pd.DataFrame(columns=['Date','Revenue'])
for row in soup.find_all("tbody")[1].find_all("tr"):
    col = row.find_all('td')
    date = col[0].text
    revenue = col[1].text
    tesla_revenue = pd.concat([tesla_revenue, pd.DataFrame({"Date":
 ↪[date],"Revenue": [revenue]})], ignore_index=True)
tesla_revenue.head()
```

[88]:

	Date	Revenue
0	2022-09-30	$21,454
1	2022-06-30	$16,934
2	2022-03-31	$18,756
3	2021-12-31	$17,719
4	2021-09-30	$13,757

Execute the following line to remove the comma and dollar sign from the `Revenue` column.

[89]:
```
tesla_revenue["Revenue"] = tesla_revenue['Revenue'].str.
 ↪replace(',|\$',"",regex=True)
```

Execute the following lines to remove an null or empty strings in the Revenue column.

[90]:
```
tesla_revenue.dropna(inplace=True)

tesla_revenue = tesla_revenue[tesla_revenue['Revenue'] != ""]
```

```
/tmp/ipykernel_299/2273276853.py:1: Pandas4Warning: The inplace keyword in
DataFrame.dropna is deprecated and will be removed in a future version (PDEP-8).
```

```
See https://pandas.pydata.org/docs/dev/whatsnew/v3.1.0.html#deprecation-inplace
for more details.
  tesla_revenue.dropna(inplace=True)
```

Display the last 5 row of the `tesla_revenue` dataframe using the `tail` function. Take a screenshot of the results.

[91]:
```
tesla_revenue.tail()
```

[91]:

	Date	Revenue
48	2010-09-30	31
49	2010-06-30	28
50	2010-03-31	21
52	2009-09-30	46
53	2009-06-30	27

0.4 Question 3: Use yfinance to Extract Stock Data

Using the `Ticker` function enter the ticker symbol of the stock we want to extract data on to create a ticker object. The stock is GameStop and its ticker symbol is `GME`.

[92]:
```
gameStop = yf.Ticker("GME")
```

Using the ticker object and the function `history` extract stock information and save it in a dataframe named `gme_data`. Set the `period` parameter to `"max"` so we get information for the maximum amount of time.

[93]:
```
gme_data = gameStop.history(period="max")
```

Reset the index using the `reset_index(inplace=True)` function on the gme_data DataFrame and display the first five rows of the `gme_data` dataframe using the `head` function. Take a screenshot of the results and code from the beginning of Question 3 to the results below.

[94]:
```
gme_data.reset_index(inplace=True)
gme_data.head()
```

```
/tmp/ipykernel_299/4260624996.py:1: Pandas4Warning: The inplace keyword in
DataFrame.reset_index is deprecated and will be removed in a future version
(PDEP-8).
See https://pandas.pydata.org/docs/dev/whatsnew/v3.1.0.html#deprecation-inplace
for more details.
  gme_data.reset_index(inplace=True)
```

[94]:

	Date	Open	High	Low	Close	Volume	\
0	2002-02-13 00:00:00-05:00	1.620128	1.693350	1.603296	1.691666	76216000	
1	2002-02-14 00:00:00-05:00	1.712708	1.716074	1.670626	1.683251	11021600	
2	2002-02-15 00:00:00-05:00	1.683250	1.687458	1.658001	1.674834	8389600	
3	2002-02-19 00:00:00-05:00	1.666418	1.666418	1.578047	1.607504	7410400	
4	2002-02-20 00:00:00-05:00	1.615920	1.662210	1.603296	1.662210	6892800	

```
   Dividends  Stock Splits
0        0.0           0.0
1        0.0           0.0
2        0.0           0.0
3        0.0           0.0
4        0.0           0.0
```

0.5 Question 4: Use Webscraping to Extract GME Revenue Data

Use the `requests` library to download the webpage https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-PY0220EN-SkillsNetwork/labs/project/stock.html. Save the text of the response as a variable named `html_data_2`

[95]:
```
url2 = " https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/
  ↪IBMDeveloperSkillsNetwork-PY0220EN-SkillsNetwork/labs/project/stock.html"
html_data_2 = requests.get(url2).text
```

Parse the html data using `beautiful_soup` using parser i.e `html5lib` or `html.parser`.

[96]:
```
soup2 = BeautifulSoup(html_data_2,'html.parser')
```

Using `BeautifulSoup` or the `read_html` function extract the table with `GameStop Revenue` and store it into a dataframe named `gme_revenue`. The dataframe should have columns `Date` and `Revenue`. Make sure the comma and dollar sign is removed from the `Revenue` column.

Note: Use the method similar to what you did in question 2.

Click here if you need help locating the table

```
Below is the code to isolate the table, you will now need to loop through the rows and columns

soup.find_all("tbody")[1]

If you want to use the read_html function the table is located at index 1
```

[97]:
```
gme_revenue = pd.DataFrame(columns = ['Date','Revenue'])
for row in soup2.find_all('tbody')[1].find_all('tr'):
    col = row.find_all('td')
    date = col[0].text
    revenue = col[1].text
    gme_revenue = pd.concat([gme_revenue,pd.DataFrame({'Date':[date],"Revenue":
  ↪[revenue]})],ignore_index=True)
```

Remove the comma and dollar sign, an null or empty strings from the Revenue column.

[98]:
```
gme_revenue["Revenue"] = gme_revenue['Revenue'].str.
  ↪replace(',|\$',"",regex=True)
```

Display the last five rows of the `gme_revenue` dataframe using the `tail` function. Take a screenshot of the results.

[99]:
```
gme_revenue.tail()
```

[99]:

	Date	Revenue
57	2006-01-31	1667
58	2005-10-31	534
59	2005-07-31	416
60	2005-04-30	475
61	2005-01-31	709

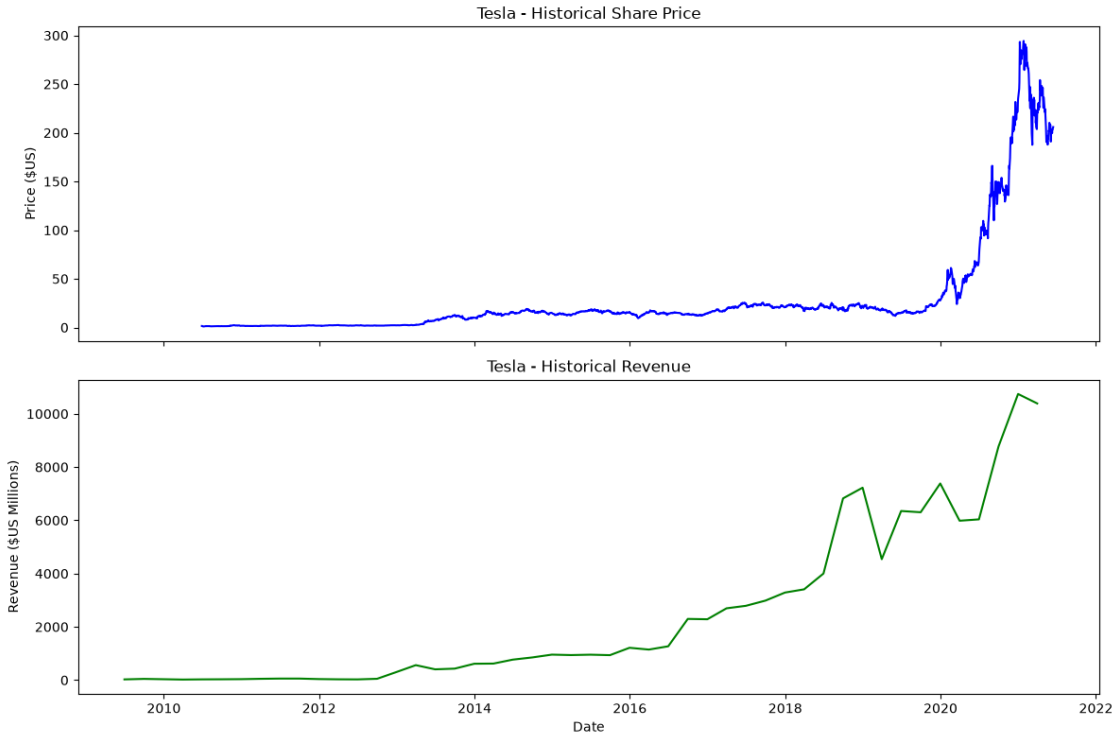
0.6 Question 5: Plot Tesla Stock Graph

Use the `make_graph` function to graph the Tesla Stock Data, also provide a title for the graph. Note the graph will only show data upto June 2021.

Hint

```
You just need to invoke the make_graph function with the required parameter to print the graphs
```

[100]:
```
make_graph(tesla_data,tesla_revenue,'Tesla')
```



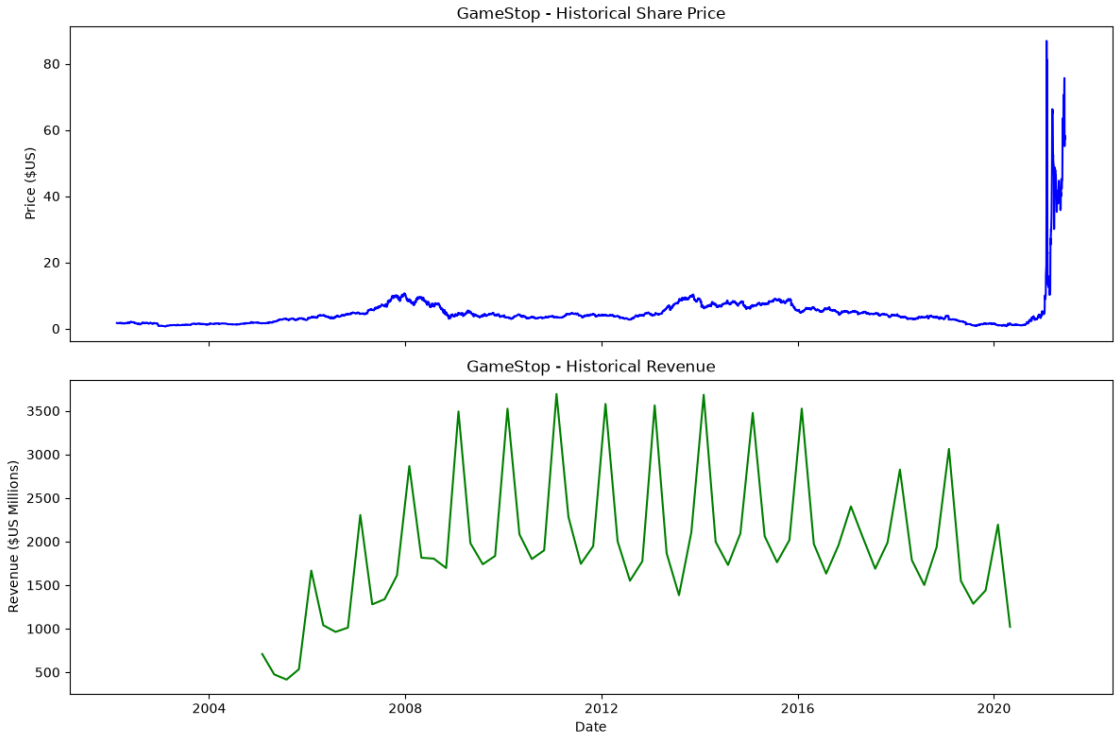
0.7 Question 6: Plot GameStop Stock Graph

Use the `make_graph` function to graph the GameStop Stock Data, also provide a title for the graph. The structure to call the `make_graph` function is `make_graph(gme_data, gme_revenue, 'GameStop')`. Note the graph will only show data upto June 2021.

Hint

`You just need to invoke the make_graph function with the required parameter to print the graphs`

[101]:
```
make_graph(gme_data,gme_revenue,'GameStop')
```



About the Authors:

Joseph Santarcangelo has a PhD in Electrical Engineering, his research focused on using machine learning, signal processing, and computer vision to determine how videos impact human cognition. Joseph has been working for IBM since he completed his PhD.

Azim Hirjani

0.8 Change Log

Date (YYYY-MM-DD)	Version	Changed By	Change Description
2022-02-28	1.2	Lakshmi Holla	Changed the URL of GameStop
2020-11-10	1.1	Malika Singla	Deleted the Optional part

Date (YYYY-MM-DD)	Version	Changed By	Change Description
2020-08-27	1.0	Malika Singla	Added lab to GitLab

Copyright © 2020 IBM Corporation. All rights reserved.